Neural Networks Training and Testing Hands-On Neural Network Training, Experiment Logging, and Loss Diagnosis

Sayat, Lyona Yvonne A.
Derpatment of Computer Studies and Engineering
Jose Rizal University
Mandaluyong City, Philippines
lyonnayvonne.sayat@my.jru.edu

***Abstract*—This paper analyzes a series of Keras text classification experiments conducted to compare code based optimization with a no code baseline and to examine model generalization through loss curves. The PT P1 baseline achieved 80.00 percent validation accuracy and a weighted F1 score of 0.7920, while the initial Keras baseline achieved 61.54 percent accuracy and a weighted F1 score of 0.5874. Across EXP 01 to EXP 05, reducing capacity and learning rate produced underfitting, whereas high capacity with an aggressive learning rate produced clear overfitting. Dropout and reduced hidden dimension improved training stability but did not always maximize F1 score. Additional training data produced a stronger result in EXP 05B, which reached a final weighted F1 score of 0.8182 and exceeded the PT P1 benchmark. The analysis also maps the notebook implementation to forward propagation, loss calculation, backpropagation, and parameter updates.**

Keywords—text classification, neural network, hyperparameter optimization, overfitting, underfitting, dropout, Keras, generalization

I. INTRODUCTION

The objective of the exercise was not only to train a text classification network, but also to understand how architectural capacity, optimization settings, regularization, and data characteristics jointly affect generalization. The PT-P1 no-code model was used as the benchmark. The notebook records a validation accuracy of 80.00 percent and a weighted F1 score of 0.7920 for that baseline. In contrast, the first Keras implementation, labeled EXP-01-KERAS-BASELINE, achieved 61.54 percent validation accuracy, a final validation loss of 0.8154, and a weighted F1 score of 0.5874. Thus, directly implementing the network in Python did not immediately outperform the automated no-code workflow.

The Keras pipeline used a fixed maximum sequence length of 20 tokens, a vocabulary limit of 1000 words, and a 70 percent training, 15 percent validation, and 15 percent testing split. The text was converted to integer sequences using a frequency based vocabulary. These sequences were then processed by an Embedding layer, GlobalAveragePooling1D, a Dense hidden layer, Dropout, and a Softmax output layer. The model was compiled with Sparse Categorical Crossentropy and Adam or SGD, depending on the configuration. The notebook also fixed NumPy and TensorFlow random seeds at 42 to improve reproducibility.

From a practical perspective, the initial gap between PT-P1 and EXP-01 is important. The no-code tool was already benefiting from its own automated heuristics, while the Keras version exposed the user to every major design decision. The lower EXP-01 score therefore demonstrates that explicit control does not automatically produce a better model; it creates the opportunity for better control, but the quality of the chosen settings remains decisive

II. EXPERIMENTAL METHODOLOGY AND HYPERPARAMETER CONFIGURATIONS

Five principal Keras runs, EXP-01 through EXP-05, were used to study the behavior of the training and validation losses. The experiment sequence was deliberately varied across the three hyperparameter pillars: architecture, optimization, and regularization. EXP-02 reduced hidden capacity and the learning rate, EXP-03 sharply increased both, EXP-04 introduced stronger dropout while reducing hidden capacity, and EXP-05 increased the embedding size and training duration while using a lower learning rate and moderate dropout.

TABLE I. SUMMARY OF EXPERIMENT CONFIGURATIONS AND VALIDATION RESULTS

Experiment	Embed	Hidden	LR	Dropout	Val Acc.	Weighted F1
PT-P1 No-Code Baseline	-	-	-	-	80.00%	0.7920
EXP-01 Keras Baseline	32	64	0.005	0.1	61.54%	0.5874
EXP-02	32	16	0.0001	0.0	38.46%	0.3582
EXP-03	32	256	0.05	0.0	69.23%	0.6374
EXP-04	32	128	0.005	0.4	61.54%	0.5917
EXP-05	64	128	0.001	0.2	53.85%	0.5201
EXP-05A Additional Data	256	256	0.001	0.30	72.73%	0.7273
EXP-05B Additional Data	32	256	0.0007	0.15	81.82%*	0.8182

Note: the asterisk on EXP-05B marks the final validation accuracy reported in the experiment notes. The notebook also reports a higher validation accuracy of 90.91 percent at Epoch 12 before declining to 81.82 percent by Epoch 15. For the main

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

experiment ranking, the weighted F1 score is treated as the more direct comparison metric because it is explicitly logged for every completed run.

III. Prepare Your Paper Before Styling

The notebook explicitly records both training loss and validation loss for every epoch and uses these histories to construct the loss curves. This comparison is the main diagnostic tool for identifying whether the network is underfitting, overfitting, or learning in a more controlled manner. The most important pattern is not simply whether the training loss becomes small, but whether validation loss improves at a compatible rate.

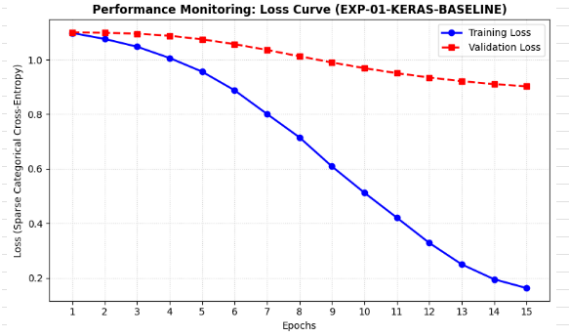


Fig. 1. EXP-01-KERAS-BASELINE: training and validation loss curves.

EXP-01 showed a steady reduction in training loss from 1.0981 to 0.1196 while validation loss declined more gradually from 1.0977 to 0.8154. The curves did not diverge sharply upward, so the run is better interpreted as a reasonable fit with a persistent generalization gap rather than severe overfitting.

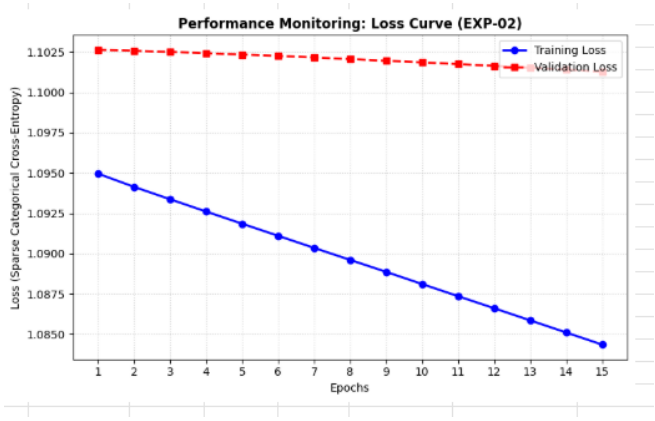


Fig. 2. EXP-02: training and validation loss curves.

EXP-02 is the clearest underfitting case. Training loss moved only from 1.0984 to 1.0925 and validation loss remained around 1.10. The model had only 16 hidden neurons and a learning rate of 0.0001, so it made limited progress in learning useful decision boundaries within 15 epochs.

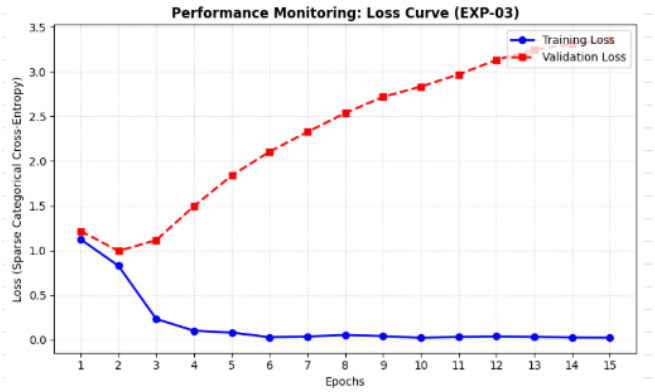


Fig. 3. EXP-03: training and validation loss curves.

EXP-03 provides the strongest evidence of overfitting. Training loss fell rapidly from 1.1208 to 0.0324, while validation loss decreased early but then rose from 1.0060 at Epoch 2 to 2.4713 at Epoch 15. Training accuracy reached 98.33 percent, while validation accuracy remained much lower and fluctuated. The combination of 256 hidden neurons, a learning rate of 0.05, and zero dropout created a highly aggressive fitting regime.

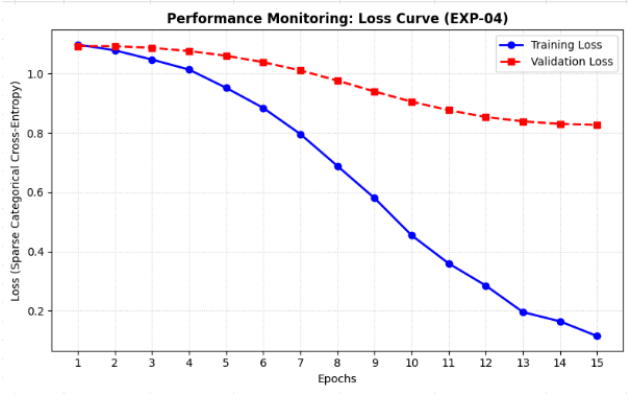


Fig. 4. EXP-04: training and validation loss curves.

EXP-04 produced a more controlled curve. Training loss decreased from 1.0975 to 0.1066 and validation loss finished at 0.8094 without the late upward explosion seen in EXP-03. The hidden dimension was reduced to 128 and dropout increased to 0.4. This supports the interpretation that regularization reduced the tendency to memorize the training set.

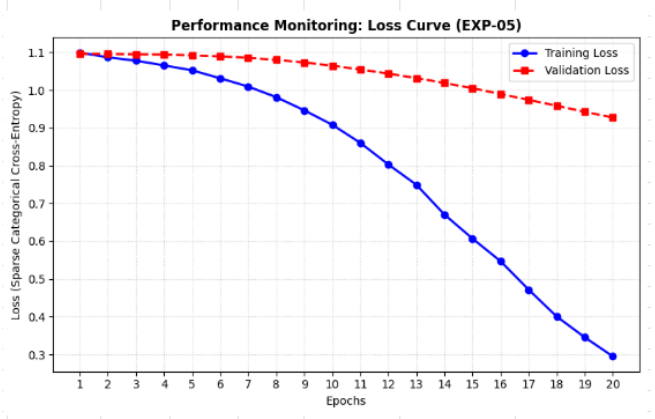


Fig. 5. EXP-05: training and validation loss curves.

EXP-05 used a 64 dimensional embedding, 128 hidden neurons, a 0.001 learning rate, 0.2 dropout, a batch size of 16, and 20 epochs. Training loss decreased to 0.3034 while validation loss reached 0.9245. Both losses continued downward, but the separation between them increased, indicating that the network was learning the training set more strongly than the validation set.

IV. Discussion & Debrief Answers

A. Code-Based Optimization Versus No-Code Heuristics

The main difference between the two approaches is the location of control. In the PT-P1 no-code workflow, the tool handled many modeling decisions through automated heuristics. The user mainly observed the resulting performance. In the Keras implementation, the user directly selected the embedding size, hidden dimension, learning rate, batch size, number of epochs, optimizer, and dropout rate. The notebook therefore exposes the full experimental design instead of hiding the tuning process behind automation.

Direct manipulation is valuable because every change can be tied to a hypothesis. For example, EXP-02 intentionally reduced the hidden dimension to 16 and the learning rate to 0.0001, while EXP-03 moved in the opposite direction by using 256 hidden neurons and a learning rate of 0.05. This made the resulting changes in model behavior observable rather than accidental. The trade-off is that explicit control increases both flexibility and responsibility. A poor learning rate can make training extremely slow or unstable, an excessively large hidden layer can encourage memorization, and too much dropout can suppress useful learning.

The results demonstrate this trade-off directly. EXP-01 was 18.46 percentage points lower than PT-P1 in validation accuracy and had a weighted F1 score approximately 0.2046 below the no-code benchmark. However, manual control made it possible to identify why certain configurations failed and to test targeted corrections. Thus, no-code automation is efficient for obtaining a strong starting point, whereas code based optimization is more useful for systematic experimentation, reproducibility, and understanding the causes of model behavior.

B. Diagnosing Model Fit Through Loss Curves and Regularization

Underfitting was indicated when both training and validation losses remained high and failed to improve substantially. EXP-02 illustrates this pattern clearly: training loss only moved from 1.0984 to 1.0925 and validation loss remained near 1.10. The model was not learning the training data effectively enough, so its validation performance could not become strong either. In this case, the small hidden dimension of 16 and extremely low learning rate of 0.0001 limited effective learning capacity.

Overfitting was indicated by a continuing decrease in training loss combined with a substantial increase in validation loss. EXP-03 is the strongest example. Its training loss dropped to 0.0324, but validation loss increased to 2.4713 by the final epoch. This means the model continued to become better at reproducing the training samples while becoming less reliable on unseen samples. The large hidden layer, aggressive learning rate, and absence of dropout made this behavior much more pronounced.

Introducing dropout changed that behavior. EXP-04 used 128 hidden neurons and a dropout rate of 0.4, and its validation loss continued to decrease to 0.8094 instead of diverging upward. The result suggests that stronger regularization reduced memorization. However, the final weighted F1 score of 0.5917 was only slightly above the EXP-01 score of 0.5874 and remained below EXP-03's 0.6374. Therefore, regularization improved stability without automatically producing the highest classification score.

EXP-05 further shows why model fit should be diagnosed from curves rather than a single metric. Training loss decreased steadily and validation loss also declined, so there was no severe late-stage validation-loss explosion. Nevertheless, the gap between training and validation behavior remained noticeable. This is best characterized as relatively stable learning with some overfitting tendency rather than a severe failure of generalization.

C. Data Volume and Diversity Versus Architectural Capacity

When hyperparameter tuning on the original dataset did not surpass the PT-P1 benchmark, the next step in the notebook was to use an additional training dataset. EXP-05A and EXP-05B changed the data source to Additional_Training_Data.csv while retaining the same 20 token sequence length and vocabulary limit of 1000. This is important because simply enlarging a network does not create new information. Additional samples can give the model more examples of how language patterns map to classes, while greater diversity can reduce the temptation to memorize a small set of repeated examples.

EXP-05A reached a final validation accuracy of 72.73 percent and a weighted F1 score of 0.7273. EXP-05B then used the additional data with a 32 dimensional embedding, 256 hidden neurons, a 0.0007 learning rate, and 0.15 dropout. Its final weighted F1 score was 0.8182, exceeding the PT-P1 F1 score of 0.7920 by 0.0262. The experiment notes also report that validation accuracy peaked at 90.91 percent at Epoch 12 before falling to 81.82 percent by Epoch 15. This indicates that the extra data increased the amount of useful information available to the model, while monitoring validation performance prevented the best intermediate result from being overlooked.

The broader lesson is that network capacity must be matched to the amount and diversity of representative data. A high-capacity network trained on a small or repetitive dataset has enough flexibility to memorize examples, which is precisely what the EXP-03 curve suggests. Conversely, a small network or very conservative optimizer can fail to capture meaningful patterns even when the task contains useful signal, as in EXP-02. Data expansion therefore complements, rather than replaces, architectural tuning: more samples can support a larger model, but only when those samples add meaningful variation and coverage.

D. Pipeline Mapping: Forward Pass, Loss, Backward Pass, and Weight Updates

The notebook implements the lecture architecture as a sequential feed-forward pipeline. In the baseline code cell,

the Embedding layer is created in the architecture block, followed by GlobalAveragePooling1D, Dense, Dropout, and the Softmax output layer. The model is then compiled with Sparse Categorical Crossentropy, an optimizer, and accuracy. The actual repeated training cycle is invoked through model.fit.

More specifically, the Embedding operation occurs at the line beginning model.add(Embedding(...)), followed by GlobalAveragePooling1D and the Dense hidden layer. The Embedding layer converts integer token IDs into trainable dense vectors. GlobalAveragePooling1D then aggregates the sequence of token vectors into a single fixed length feature representation. This is the feature extraction stage. Instead of manually calculating features such as word counts or handcrafted statistics, the network learns the embedding values and uses the pooled representation as the input to the classifier.

Forward Pass and Loss Calculation: these operations occur inside the model.fit training loop after each batch of X_train and y_train is supplied. The model propagates the batch through Embedding, pooling, Dense, Dropout, and Softmax to produce class probabilities. Sparse Categorical Crossentropy then compares those probabilities with the true integer labels. The notebook explicitly explains that model.fit performs the forward pass, loss calculation, backward propagation, and weight updates as part of the complete learning cycle.

Backward Pass and Weight Updates: these also occur inside model.fit. After the loss is computed, TensorFlow automatically calculates gradients through backpropagation and passes them to the configured optimizer. In the baseline, Adam is constructed from the learning rate and assigned to model.compile. Adam then applies the computed gradients to the trainable weights. The user does not need to manually write derivative equations for every layer, but the code still provides explicit control over the optimizer choice and learning rate.

Validation Monitoring: validation_data=(X_val, y_val) is passed to model.fit so that validation metrics can be calculated after each epoch without using the validation samples to update the weights. The subsequent history.history['loss'] and history.history['val_loss'] statements retrieve the recorded learning curves. Finally, model.predict(X_val), argmax, and weighted F1 calculation convert predictions into class labels and summarize validation performance.

For line mapping, the raw notebook file shows the baseline preprocessing beginning around lines 442 to 481, the architecture at lines 503 to 523, optimizer and compilation at lines 529 to 539, and the main training loop at lines 543 to 555. The loss and validation histories are retrieved at lines 571 to 573. These line ranges can be cited as the concrete implementation points corresponding to the lecture pipeline.

V. CONCLUSION

Validation Monitoring: validation_data=(X_val, y_val) is passed to model.fit so that validation metrics can be calculated after each epoch without using the validation samples to update the weights. The subsequent history.history['loss'] and history.history['val_loss'] statements retrieve the recorded learning curves. Finally, model.predict(X_val), argmax, and weighted F1 calculation convert predictions into class labels and summarize validation performance.

The experiments show that generalization is controlled by the interaction of architecture, optimization, regularization, and data. The PT-P1 no-code model started from a stronger validation benchmark than the initial Keras implementation, but the code based pipeline made it possible to diagnose the causes of weak performance. EXP-02 showed underfitting because the model had insufficient effective learning capacity. EXP-03 showed strong overfitting because high capacity and an aggressive learning rate allowed training loss to collapse while validation loss increased. EXP-04 demonstrated that reducing hidden capacity and adding stronger dropout produced a more stable loss relationship, although it did not maximize F1.

EXP-05 reinforced the principle that longer training and richer representations are not automatically beneficial for generalization. The expanded-data experiments were more encouraging: EXP-05A improved over the earlier EXP-05 run, and EXP-05B ultimately achieved a weighted F1 score of 0.8182, surpassing the PT-P1 benchmark of 0.7920. The most important practical conclusion is that model improvement should be treated as an iterative process. First establish a baseline, then inspect the loss curves, change one or a small group of related variables, and use validation behavior to decide whether the change improved learning rather than merely increasing training accuracy.

Overall, the notebook demonstrates why neural network training is not simply a matter of increasing model size. The strongest configuration is the one that balances enough capacity to learn useful patterns with enough data and regularization to preserve generalization. The combination of explicit Keras control, loss-curve diagnosis, and expanded data provided a clearer explanation of model behavior than either no-code automation or training accuracy alone.

REFERENCES

[1] F. Chollet, “Training and evaluation with the built-in methods,” TensorFlow Core, 2023.

[2] X. Liang, L. Wu, J. Li, Y. Wang, Q. Meng, W. Chen, M. Zhang, and T.-Y. Liu, “R-Drop: Regularized dropout for neural networks,” 2021

[3] L. Shen, Y. Sun, Z. Yu, L. Ding, X. Tian, and D. Tao, “On efficient training of large-scale deep learning models: A literature review,” 2023